PROJECT REPORT

On

“Expression Tree Generator And Evaluator ”

[Experiential Learning]

Submitted By:

Pranjal C Parihar [CM25101]

Guided By:

Ms.Suchita Mesakar



DEPARTMENT OF CSE (AI &ML)

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH, NAGPUR.

(An Autonomous Institute,AffiliatedtoRTMNU, Nagpur)

2026-2027

© S.B.J.I.T.M.RNagpur

INDEX

INDEX		i
CHAPTER 1	INTRODUCTION	1
CHAPTER 2	METHODOLOGY	2-3
CHAPTER 3	TOOLS/PLATFORMS	4
CHAPTER 4	DESIGN & IMPLEMENTATION	5-8
	4.1 ALGORITHM	
	4.2 FLOWCHART	
	4.3 SOURCE CODE	
CHAPTER 5	RESULT & DISCUSSION	9-11
	5.1 OUTPUT	
	5.2 DISCUSSION	
	5.3 APPLICATION	
CHAPTER 6	CONCLUSION	12
	REFERENCES	13

CHAPTER 1
INTRODUCTION

Introduction:

The Expression Tree Generator and Evaluator is a Python-based project designed to construct and evaluate arithmetic expressions using a tree data structure. An expression tree is a binary tree in which the leaf nodes represent operands, such as numbers, and the internal nodes represent operators, such as addition, subtraction, multiplication, and division. This project helps users understand how arithmetic expressions can be represented in the form of a tree and evaluated efficiently.

The project accepts an arithmetic expression as input, processes the expression, and generates its corresponding expression tree. It then evaluates the expression using tree traversal and recursive functions. The system helps reduce errors in manual calculations and makes it easier to understand the order of operations in complex expressions.

Background:

Arithmeticexpressions are commonly used in calculators, programming languages, and mathematical applications. Evaluating complex expressions manually can be difficult because operators follow different precedence rules. Expression trees provide a structured method for representing expressions and evaluating them in the correct order.

Features:

- Accepts arithmetic expressions as input.
- Processes expressions and constructs an expression tree.
- Represents operands and operators using tree nodes.
- Evaluates expressions using recursive functions.
- Demonstrates binary tree traversal.

CHAPTER 2
METHODOLOGY

The methodology of the Expression Tree Generator and Evaluator project explains the steps used to accept an arithmetic expression, convert it into an expression tree, and calculate the final result. The project is developed using Python programming and the concept of data structures.

Step 1: Input Expression
The user enters an arithmetic expression containing numbers and operators such as +, -, *, and /.
Step 2: Expression Processing
The program reads the input expression and processes its elements to understand the operands and operators.
Step 3: Expression Tree Construction
The program arranges the operands and operators in a binary tree structure. The operators are stored in internal nodes, while the operands are stored in leaf nodes.
Step 4: Tree Traversal
The program uses tree traversal techniques to access and process the nodes of the expression tree.
Step 5: Expression Evaluation
The program recursively evaluates the expression tree by performing arithmetic operations in the required order.
Step 6: Display Output
Finally, the program displays the evaluated result to the user through the application interface.

Step 7: Result Verification
The program checks the evaluated result to ensure that the arithmetic calculation has been performed correctly.
Step 8: Operator Precedence
The expression is processed according to the order of operations so that the calculation gives the correct result.
Step 9: Recursive Function Execution
The program uses recursive functions to process the tree nodes and evaluate the expression.
Step 10: Tree Visualization
The expression tree is displayed visually, if supported by the application, to help the user understand the structure of the expression.
Step 11: User Interface
The application provides an interface where users can enter expressions and view the generated tree or calculated result, depending on the implemented features.
Step 12: Error Handling
The program handles invalid expressions or incorrect inputs where error-handling functionality is implemented.
Step 13: Output Display
The final calculated result is displayed to the user in an understandable format.
Step 14: Testing
The application is tested with different arithmetic expressions to check whether the expression tree and calculated results work correctly.
Step 15: Final Execution
After testing, the program is executed to demonstrate the complete process of expression input, tree construction, evaluation, and output display.

CHAPTER 3
TOOLS/PLATFORMS

3.1 SOFTWARE REQUIREMENT

1.Python:Pythonisthemainprogramming language used to develop the project. It helps in processing arithmetic expressions, constructing expression trees, and evaluating the final result.

2. Visual Studio Code (VS Code): It can be used as a code editor to write, edit, and manage the Python program.

3. Streamlit: If the project uses Streamlit, it provides a simple web-based interface for entering arithmetic expressions and displaying the output.

4. Binary Tree Data Structure: A binary tree is used to represent arithmetic expressions. Operators are stored in internal nodes, and operands are stored in leaf nodes.

5. Recursion: Recursive functions are used to process the tree nodes and calculate the expression's result.

6. Web Browser: If the application runs through a web interface, a browser can be used to access and interact with the application.

7. Operating System: The project can be developed and executed on a compatible operating system, such as Windows or Linux, with Python installed.

These tools and platforms help in developing, running, and testing the Expression Tree Generator and Evaluator application.

- Python Programming Language
- Visual Studio Code (VS Code)
- Streamlit Framework
- Binary Tree Data Structure
- Recursion
- Tree Traversal Techniques
- Web Browser
- Operating System
- Testing and Debugging
- Input Validation
- Arithmetic Operators
- User-Friendly Interface

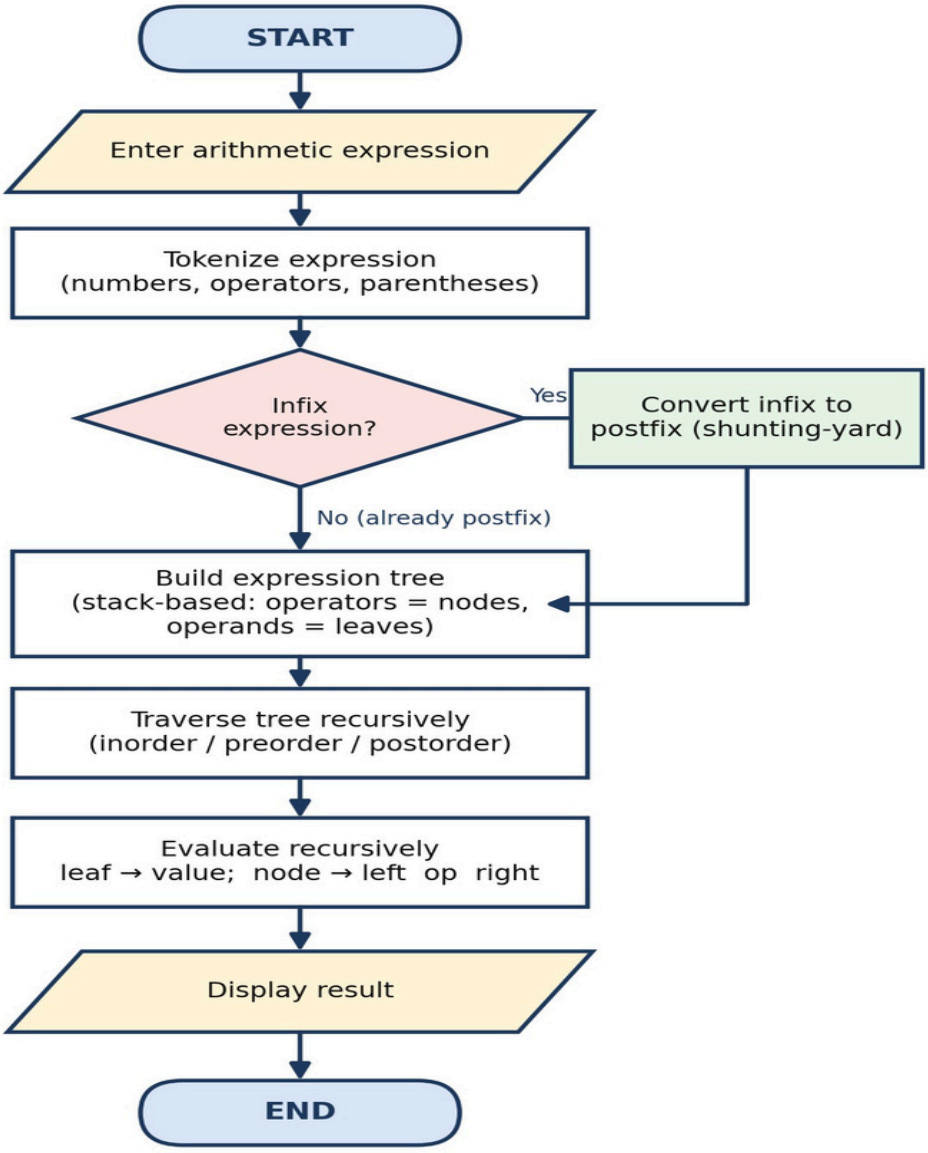
Group the topics into clear categories
Add one-line explanations for each topic
Correct and standardize the bullet formatting

CHAPTER 4
DESIGN & IMPLEMENTATION

4.1 ALGORITHM

○ Step 1: Start the application.
○ Step 2: Display the user interface.
○ Step 3: Accept an arithmetic expression from the user.
○ Step 4: Read the entered expression.
○ Step 5: Check the expression for valid input.
○ Step 6: Identify the numbers and arithmetic operators.
○ Step 7: Process the expression according to its syntax.
○ Step 8: Create nodes for the operands and operators.
○ Step 9: Construct the expression tree using the nodes.
○ Step 10: Arrange operators in internal nodes and operands in leaf nodes.
○ Step 11: Traverse the expression tree using recursion.
○ Step 12: Evaluate the left subtree.
○ Step 13: Evaluate the right subtree and perform the required arithmetic operation.
○ Step 14: Display the generated expression tree and the calculated result.
○ Step 15: End the program.

4.2 FLOWCHART



Expression Tree Generator and Evaluator

Flowchart

4.3 SOURCE CODE

```
# Expression Tree Generator and Evaluator
# Builds an expression tree from an arithmetic expression and evaluates it # using
recursive tree traversal. Operators: + - * / ^ and ( )
import re

PREC = {'+': 1, '-': 1, '*': 2, '/': 2, '^': 3} # operator precedence
RIGHT = {'^'}                                   # right-associative ops


class Node:
    """Tree node: operators are internal nodes, operands are leaves.""" def
    __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

    def is_leaf(self):
        return self.left is None and self.right is None


# ---------- 1. Tokenize ----------
def tokenize(expr):
    tokens = re.findall(r'\d+\.?\d*|\.\d+|[-+*/^()]|\S', expr)
    for t in tokens:
        if not (re.fullmatch(r'\d+\.?\d*|\.\d+', t) or t in PREC or t in '()'):
            raise ValueError(f"Invalid character: '{t}'")
    return tokens


# ---------- 2. Infix -> Postfix (shunting-yard) ----------
def to_postfix(tokens):
    out, ops = [], []
    for t in tokens:
        if t in PREC:
            while (ops and ops[-1] != '(' and
                   (PREC[ops[-1]] > PREC[t] or
                   (PREC[ops[-1]] == PREC[t] and t not in RIGHT))):
                out.append(ops.pop())
            ops.append(t)
```

```
        elif t == '(':
                ops.append(t)
            elif t == ')':
                while ops and ops[-1] != '(':
                    out.append(ops.pop())
                if not ops:
                    raise ValueError('Mismatched parentheses')
                ops.pop()
            else:
                out.append(t)
        while ops:
            if ops[-1] == '(':
                raise ValueError('Mismatched parentheses')
            out.append(ops.pop())
        return out
def build_tree(postfix):
    stack = []
    for token in postfix:
        node = Node(token)
        if token in PREC:                    # operator
            if len(stack) < 2:
                raise ValueError('Invalid expression (missing operand)')
node.right = stack.pop()
            node.left = stack.pop()
        else:                                # operand
            try:
                float(token)
            except ValueError:
                raise ValueError(f"Invalid token: '{token}'")
        stack.append(node)
    if len(stack) != 1:
        raise ValueError('Invalid expression')
    return stack[0]
            # ---------- 4. Recursive traversals ----------
            def inorder(n):
                if n.is_leaf():
                    return n.value
                return '(' + inorder(n.left) + '' + n.value + ' ' +
            inorder(n.right) + ')'
    def preorder(n):
        if n is None:
            return []
        return [n.value] + preorder(n.left) + preorder(n.right)
    def postorder(n):
        if n is None:
            return []
        return postorder(n.left) + postorder(n.right) + [n.value]
```

CHAPTER 5
RESULT & DISCUSSION

5.1 OUTPUT

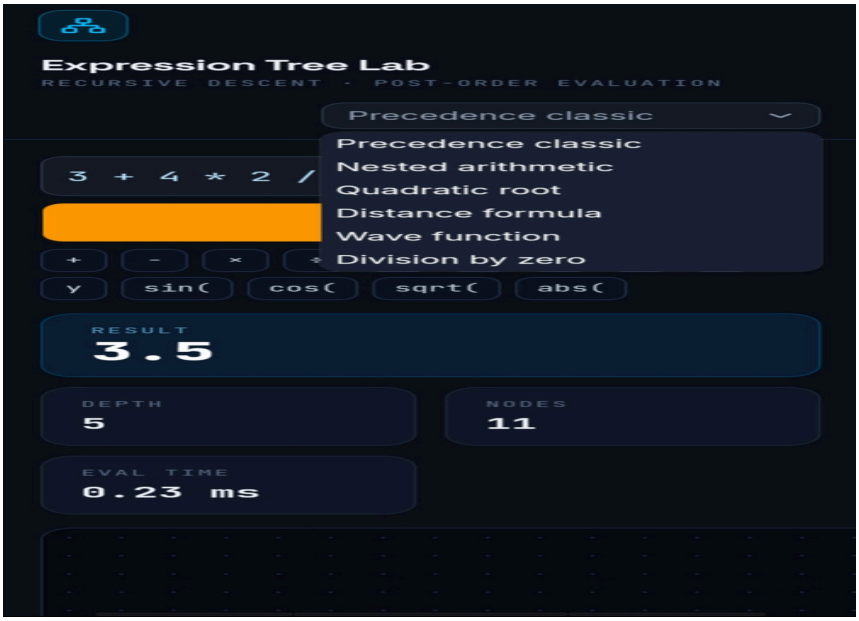


Fig. 5.1.1: Expression Tree Lab (web interface)

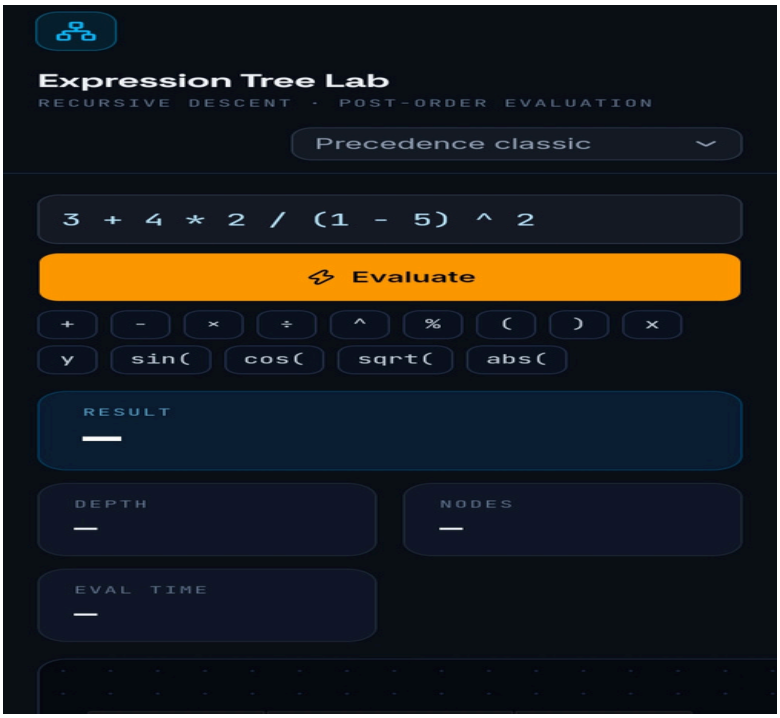


Fig. 5.1.2: Expression Tree Lab

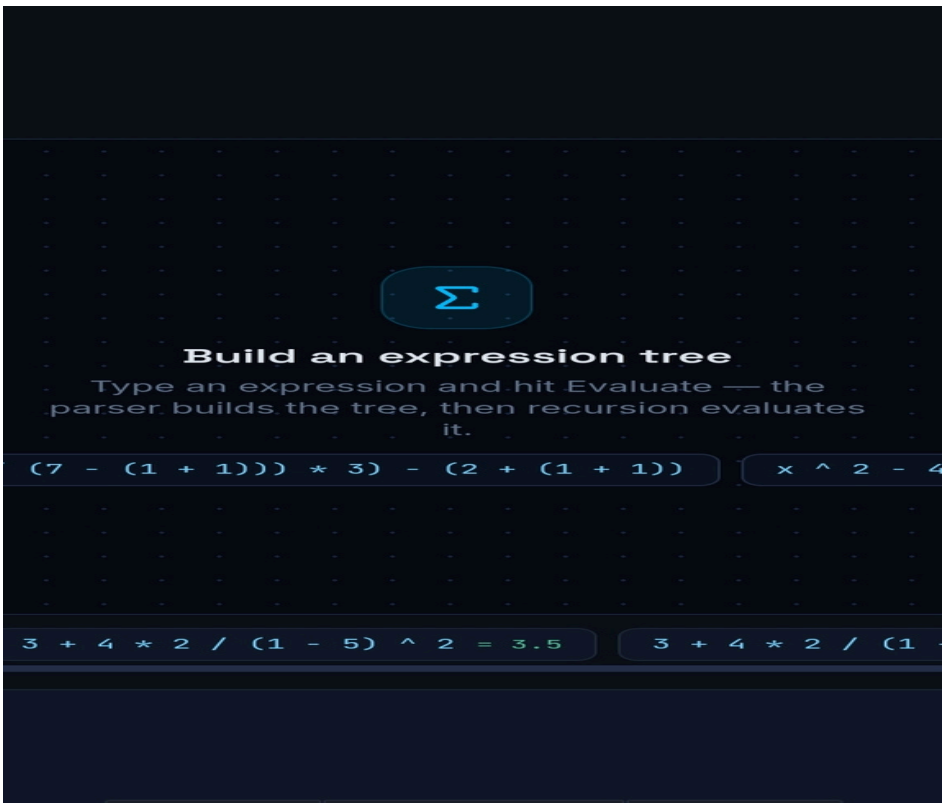


Fig. 5.1.3: Build an expression tree

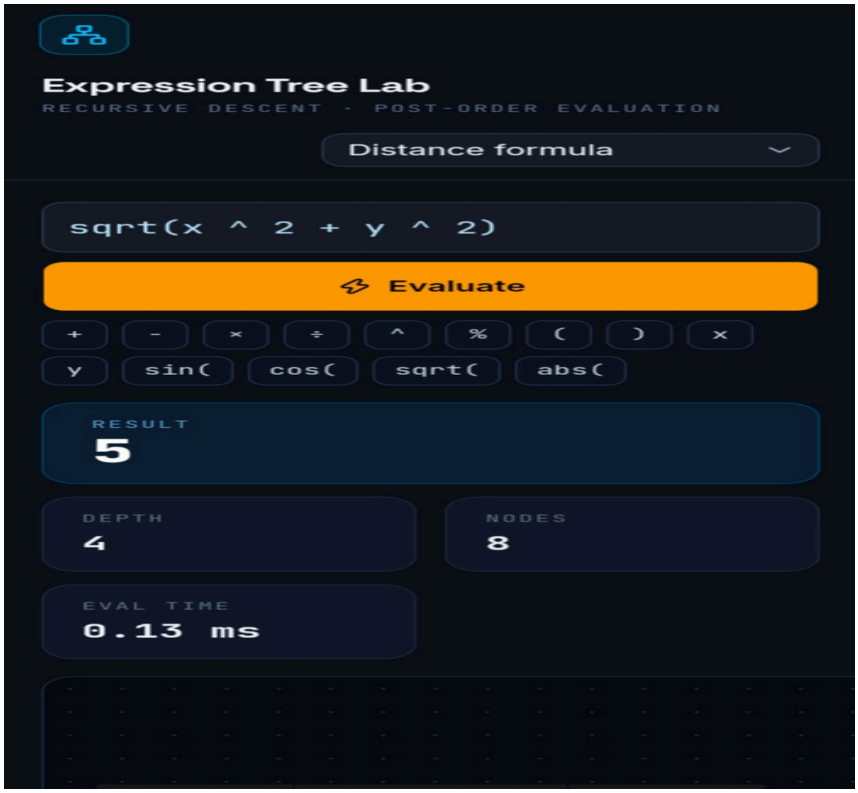


Fig. 5.1.4: Result Output

5.2 DISCUSSION

- An expression tree is a binary tree in which leaves hold operands (numbers or variables) and internal nodes hold operators, so that the structure of the tree itself encodes precedence and grouping. A generator builds this tree from an expression: for postfix input it scans left to right, pushing a node for each operand and, for each operator, popping two nodes as its right and left children and pushing the new subtree back, leaving the root as the last item on the stack; for infix input it first converts to postfix with the Shunting-Yard algorithm (or uses a recursive-descent parser), and for prefix input it scans right to left with the pop order reversed. An evaluator then computes the value with a post-order traversal, recursively evaluating the left and right subtrees and applying the node's operator, so (3 + 4) * 2, which becomes postfix 3 4 + 2 *, evaluates to 7 × 2 = 14. The same tree yields all three notations through traversal: in-order gives infix (with parentheses added), pre-order gives prefix, and post-order gives postfix. Both building and evaluating take O(n) time, with O(h) space for recursion, where h is the tree height (O(n) in the worst case). The main advantage is that precedence and associativity are resolved at build time, so evaluation needs no parentheses or extra rules, and the design extends naturally to variables, unary functions like sin and log, constant folding, symbolic

5.3 APPLICATION

- Compilers and interpreters: The parser turns source code into an abstract syntax tree (a
- generalized expression tree). Later stages use it for type checking, optimization (
- constant folding, common subexpression elimination), and code generation.
- Calculators: Scientific and graphing calculators parse typed input into a tree, then
- evaluate it, handling precedence and brackets correctly.
- Spreadsheets: Cell formulas like =A1*B1+C1 are stored as trees, so values can be recomputed when referenced cells change.
- Symbolic mathematics: Tools like Mathematica, SymPy, and Maple represent expressions as trees to simplify, expand, factor, differentiate, and integrate them.
- Database query optimizers: SQL conditions and relational operations are held as trees, which the optimizer rewrites into cheaper equivalent plans.

CHAPTER 6
CONCLUSION

Expression trees give a simple, uniform way to represent mathematical and logical expressions as data. By storing operands at the leaves and operators at internal nodes, the tree captures precedence and grouping in its structure, so evaluation becomes a clean post-order traversal with no need for parentheses or extra rules.

The same tree converts easily between infix, prefix, and postfix notations, and both building and evaluating run in linear time.

Beyond evaluation, the tree can be inspected, simplified, differentiated, and optimized, which is why the idea underlies compilers, calculators, spreadsheets, symbolic math systems, query optimizers, and machine learning frameworks.

A robust implementation should also handle errors such as division by zero, malformed input, and unary operators.

Overall, the expression tree generator and evaluator is a foundational data structure application that links parsing theory with practical software design.

REFERENCES

[1]. Python Software Foundation, Python 3 Documentation. Publication date: Continuously updated. Available at: https://docs.python.org/3/

[2]. Python Software Foundation, Python Regular Expression (re) Documentation. Available at: https://docs.python.org/3/library/re.html

[3]. Python Software Foundation, collections — Container Datatypes (deque,used for stacksand queues). Availableat: https://docs.python.org/3/library/collections.html

[4]. Python Software Foundation, Data Structures (Using lists as stacks). Available at: https://docs.python.org/3/tutorial/datastructures.html

[5]. Python Software Foundation, operator — Standard Operators as Functions. Available at: https://docs.python.org/3/library/operator.html

[6]. Python Software Foundation, ast — Abstract Syntax Trees. Available at: https://docs.python.org/3/library/ast.html